

Coaching Qwen3-Coder-30B to Think Like a CodeClash Arena Agent

Ivy, Ning Zhang

Department of Computer Science, Stanford University



Problem and Motivation

Question. How can we improve the long-horizon behavior of a weak open-weight coding agent?

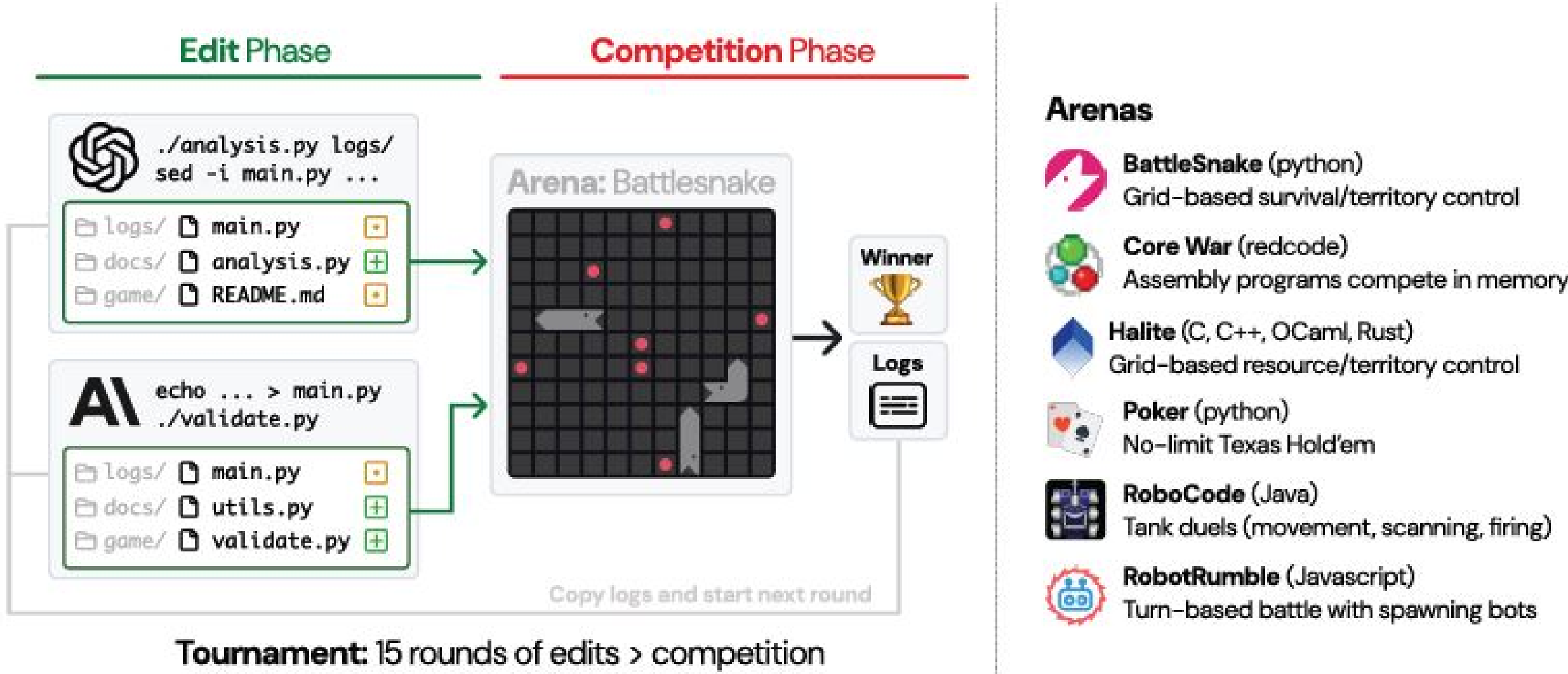
Motivation. In practical coding workflows, weaker or open-weight agents often struggle to reliably interpret user intent, follow multi-step requests, and remain stable over repeated interactions, while stronger frontier agents handle the same settings much more consistently.

Why long-horizon coding is hard for coding agents?The difficulty lies in sequential decision-making: a coding agent must use feedback from earlier rounds to diagnose failures, maintain consistency across edits, and select actions that are beneficial for future rounds rather than merely plausible in the current one

Why CodeClash? CodeClash is a strong testbed for this question because it evaluates coding agents in a truly long-horizon setting: agents repeatedly edit a private repository, compete in an executable arena, and use logs and round outcomes as feedback for later rounds. This is much closer to real agentic coding than one-shot benchmarks.

Task Setting: BattleSnake Arena

We focus on the BattleSnake arena in CODECLASH, where agents repeatedly edit a private repository, compete in an executable environment, and receive logs and round outcomes as feedback for future rounds. Rather than measuring one-shot code generation, this setting evaluates whether an agent can adapt its behavior across repeated edit–feedback cycles. Our Goal is to coach the weaker coding agent think strategically and improve it with distilled knowledge from stronger agents. Since the Qwen family ranks near the bottom in the original benchmark, Qwen3-Coder-30B is a natural open-weight case study for closing part of that gap.



Arenas

- BattleSnake** (python)
Grid-based survival/territory control
- Core War** (redcode)
Assembly programs compete in memory
- Halite** (C, C++, OCaml, Rust)
Grid-based resource/territory control
- Poker** (python)
No-limit Texas Hold'em
- RoboCode** (Java)
Tank duels (movement, scanning, firing)
- RobotRumble** (Javascript)
Turn-based battle with spawning bots

Observed Failure Modes of Base Qwen3-Coder-30B

Qwen3-Coder-30B shows recurring failures in long-horizon arena interaction. These errors are not random: they often arise from unsafe large-range edits, weak intermediate validation, and limited use of diagnostic feedback across rounds.

Failure	Typical triggering action
Startup block missing	Full-file overwrite of <code>main.py</code> or truncate-to-EOF edits that accidentally remove the required startup entrypoint
Syntax / structure broken	Large-range delete-and-replace edits or partial rewrites that introduce indentation drift or broken function structure
Malformed server output	Unsafe rewrites that preserve compilation but cause malformed metadata or non-JSON server responses at runtime

Challenge. Vanilla instruction tuning provides only weak action-level control: offline supervised finetuning does not directly teach the model whether a generated action is protocol-legal or strategically useful for future rounds. As a result, ordinary vanilla/self-play SFT does not reliably eliminate these failures, which motivates more structured post-training signals.

Approach Overview

We study two post-training signals on top of the same assistant-only SFT framework: **ReAct SFT** for stronger competitive reasoning, and **TQ-SFT** for safer edit–validate behavior.

For a serialized trajectory $t_{1:L}$ and assistant-token set Y , we optimize:

$$\mathcal{L}_{\text{SFT}} = - \sum_{j=1}^L \mathbf{1}[j \in Y] \log p_{\theta}(t_j \mid t_{<j}).$$

Implementation: QLoRA/LoRA, 4-bit backbone, bf16 computation, assistant-only masking, and sliding windows over assistant turns.

Method 1: ReAct SFT

Motivation. Vanilla SFT often teaches the surface form of an arena response (THOUGHT + `bash`) without teaching how to reason over feedback.

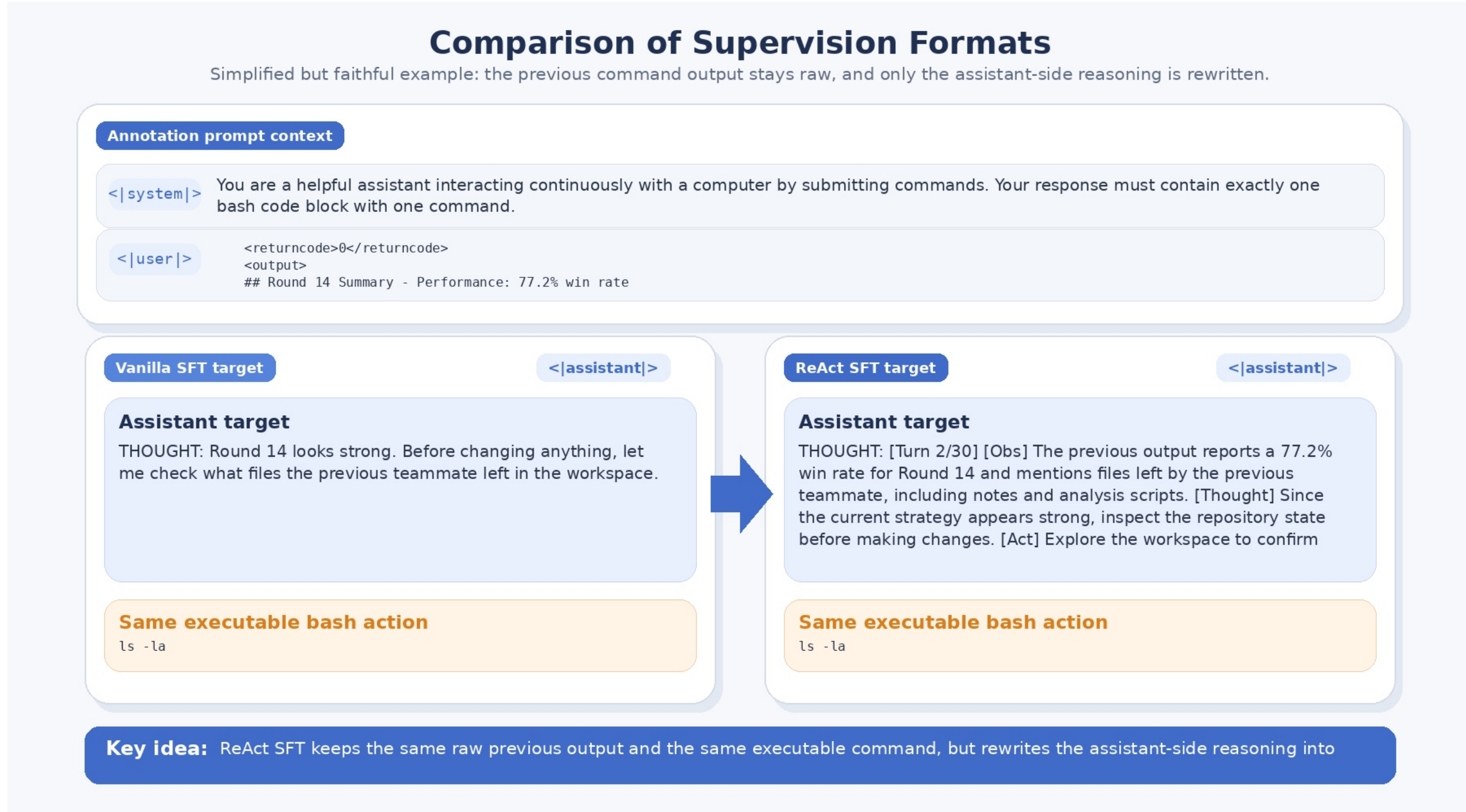
Key idea. Rewrite strong-agent trajectories into explicit

[Obs] [Thought] [Act]

while preserving the original executable bash command.

Teacher source. We use recorded Claude 4.5 trajectories and annotate them step-by-step with Claude 4.5 itself.

Goal. Make the observation–reasoning–action loop explicit, so the student can better learn how to inspect feedback, decide the next edit, and adapt across rounds.



Method 2: Trajectory-Quality Weighted SFT (TQ-SFT)

Motivation. Qwen3-Coder-30B often makes large edits without validation and submits broken code.

Key idea. We derive a trajectory-quality score from coarse action patterns such as *modify* → *check*, unchecked modification, and submit-after-validation behavior, then use it to reweight training examples:

$$L_{\text{TQ-SFT}} = \frac{\sum_i w_i \ell_i}{\sum_i w_i}.$$

Goal. Encourage safer *modify-then-check* behavior and reduce invalid submissions.

Data and Experimental Setup

We use the BattleSnake arena from CODECLASH to evaluate model performance. Our training data consists of recorded arena trajectories from the original benchmark and additional self-play data generated by running Qwen3-Coder-30B against itself. All variants start from Qwen3-Coder-30B and share the same QLoRA setup: learning rate 1×10^{-5} , batch size 1 with gradient accumulation 8, assistant-only loss, and sliding windows over assistant turns. Evaluation follows the original CODECLASH tournament setting in BattleSnake.

Ablations and Main Results

We first explored ReAct-style supervision with context windows of 2, 4, and 8 turns using an initial rewritten subset of about 150 trajectories. Among these early ablations, the 4-turn setting performed best: 2-turn appears too short to preserve enough arena history, while 8-turn appears too long and noisy for stable strategy transfer. TQ-SFT behaves differently: although it improves some reliability-related behaviors and reduces invalid submission rates, it still underperforms ReAct in direct competition. These results suggest that better reliability alone is not sufficient for strong tournament performance.

Matchup	Winner	Win	Lose	Tie	Primary failure side
ReAct 2-turn vs base	Base	1	12	0	both
ReAct 8-turn vs base	Base	1	12	1	both
ReAct 4-turn vs base	ReAct 4-turn	6	2	1	both
TQ-SFT vs base	Base	3	6	3	mostly B
ReAct 6-turn vs TQ-SFT	ReAct 6-turn	11	1	2	both
TQ-SFT vs Vanilla SFT	TQ-SFT	3	1	0	mostly B
ReAct 4-turn vs TQ-SFT	ReAct 4-turn	8	3	3	both

Final Tournament Comparisons

Motivated by the strong performance of ReAct 4-turn, we expanded the rewritten corpus to up to 430 trajectories and trained a 6-turn ReAct variant as our final model. This final model decisively outperforms both the untuned base model and Qwen3-Coder-Plus, while still remaining clearly below Claude 4.5.

Matchup	Winner	Win	Lose	Tie	Primary failure side
ReAct 6-turn vs base	ReAct 6-turn	9	3	1	both
ReAct 6-turn vs Qwen3-Coder-Plus	ReAct 6-turn	12	0	0	mostly B
ReAct 6-turn vs Claude 4.5	Claude 4.5	0	15	0	none

Compared with earlier ReAct variants and TQ-SFT, the final model also shows fewer abnormal collapse rounds.

Behavioral Interpretation and Takeaway

ReAct SFT more effectively transfers competitive behavior by making the observation–reasoning–action loop explicit. The resulting model is more likely to inspect round results, diagnose failures, and make tournament-oriented edits. **TQ-SFT** mainly improves reliability by encouraging safer modify–then–check behavior, but this signal alone does not reliably teach stronger strategic adaptation.

Action category	Base	Qwen-Coder-Plus	Claude 4.5	ReAct SFT	TQ-SFT
Round / result checks	1.22%	4.99%	31.75%	21.51%	9.93%
Edits-log checks	0.00%	0.07%	1.67%	6.69%	0.00%
Simulation slices	0.00%	0.40%	18.28%	0.00%	0.00%
Code structure checks	6.94%	14.88%	8.08%	1.16%	0.97%
Local testing	4.90%	0.35%	7.01%	11.34%	8.72%
Analysis scripts	0.00%	0.29%	11.98%	9.88%	12.59%

Percentages denote the share of assistant bash actions assigned to the selected categories, so columns do not sum to 100%.

Takeaway. Weak arena agents need more than ordinary instruction tuning. ReAct SFT improves **competitive reasoning**, while TQ-SFT improves **reliability**. Overall, stronger long-horizon coding agents require learning a **feedback-driven action policy**, not just better response imitation.